

# climatometre-rt

## Projet Climatomètre RT - Documentation Technique

BUT Réseaux et Télécommunications

[CLIQUEZ ICI POUR VOIR LES RESSOURCES DU PROJET \(RESOURCES.md\)](#)

version 1.0.0 Python 3.10+ Framework Flask Database SQLite3

### Introduction

Le **Climatomètre RT** est une application full-stack conçue pour centraliser, cartographier et analyser les données climatiques des lieux de vie des étudiants. Ce projet combine le développement d'interfaces dynamiques, la gestion de bases de données relationnelles et la communication avec des services API tiers.

### Architecture du Système

#### 1. Choix Technologiques

- **Backend** : Python avec le micro-framework **Flask**. Choisi pour sa souplesse et sa capacité à gérer des tâches asynchrones (threading).
- **Base de Données** : **SQLite3**. Un moteur robuste sans serveur, idéal pour la portabilité du projet entre collaborateurs.
- **Cartographie** : **Leaflet.js**. Utilisation de plugins de *Clustering* pour gérer la superposition des marqueurs dans les zones à forte densité étudiante (ex: résidences universitaires).
- **Frontend** : Design personnalisé en CSS3 avec l'utilisation de **Glassmorphism** (effets de flou et transparence).

#### 2. Modèle de Données (Schéma Relationnel)

L'application s'appuie sur une structure SQL normalisée pour garantir l'intégrité des données :

| Table         | Rôle                                                                              |
|---------------|-----------------------------------------------------------------------------------|
| Etudiants     | Stocke l'identité (Nom, Prénom).                                                  |
| Residences    | Stocke la localisation (Ville, Adresse, Type), liée à un étudiant.                |
| Relevés_Meteo | Stocke l'historique climatique (Température, Humidité, Ciel) lié à une résidence. |

### Analyse des Composants Logiciels

#### A. Gestion du rafraîchissement météo (Threading)

Pour assurer une expérience utilisateur fluide sans latence réseau, l'application utilise un **Thread daemon** en arrière-plan :

- **Processus** : Au lancement du serveur, une boucle infinie est initiée. Elle interroge l'API OpenWeatherMap pour chaque ville enregistrée toutes les 60 secondes.
- **Optimisation** : Les données sont stockées en base de données. L'interface utilisateur ne consulte que la base locale, évitant ainsi des appels API superflus et coûteux lors du chargement des pages.

#### B. Algorithme du Climat Médian

Plutôt qu'une simple moyenne, nous avons implémenté le calcul de la **médiane** pour éliminer l'influence des valeurs aberrantes (étudiants en stage à l'étranger ou erreurs de capteurs API). 
$$\tilde{x} = \begin{cases} x_{(\frac{n+1}{2})} & \text{si } n \text{ est impair} \\ \frac{x_{(\frac{n}{2})} + x_{(\frac{n}{2}+1)}}{2} & \text{si } n \text{ est pair} \end{cases}$$

#### C. Cartographie & Géocodage

Lors de l'ajout d'un étudiant, l'application effectue un **géocodage inverse** via l'API pour récupérer les coordonnées GPS (*lat*, *lon*). Ces coordonnées permettent à Leaflet de positionner précisément les marqueurs sur la carte de la promotion.

### Installation et Déploiement

#### 1. Clonage et Environnement

```
git clone [https://github.com/Raphoxrr28/Climatom-tre-RT.git] (https://github.com/Raphoxrr28/Climatom-tre-RT.git)
cd Climatom-tre-RT
python -m venv venv
# Activation (Windows) : venv\Scripts\activate
# Activation (Linux/Mac) : source venv/bin/activate
```

```
<style type="text/css">@media print {
  *, :after, :before {background: 0 0 !important;color: #000 !important;box-shadow: none !important;text-shadow: none !im
  a, a:visited {text-decoration: underline}
  a[href]:after {content: " (" attr(href) ")"}
}
```

```

abbr[title]:after {content: " (" attr(title) ")"}
a[href^="#"]:after, a[href^="javascript:"]:after {content: ""}
blockquote, pre {border: 1px solid #999;page-break-inside: avoid}
thead {display: table-header-group}
img, tr {page-break-inside: avoid}
img {max-width: 100% !important}
h2, h3, p {orphans: 3;widows: 3}
h2, h3 {page-break-after: avoid}
}
html {font-size: 12px}
@media screen and (min-width: 32rem) and (max-width: 48rem) {
  html {font-size: 15px}
}
@media screen and (min-width: 48rem) {
  html {font-size: 16px}
}
body {line-height: 1.85}
.air-p, p {font-size: 1rem;margin-bottom: 1.3rem}
.air-h1, .air-h2, .air-h3, .air-h4, h1, h2, h3, h4 {margin: 1.414rem 0 .5rem;font-weight: inherit;line-height: 1.42}
.air-h1, h1 {margin-top: 0;font-size: 3.998rem}
.air-h2, h2 {font-size: 2.827rem}
.air-h3, h3 {font-size: 1.999rem}
.air-h4, h4 {font-size: 1.414rem}
.air-h5, h5 {font-size: 1.121rem}
.air-h6, h6 {font-size: .88rem}
.air-small, small {font-size: .707em}
canvas, iframe, img, select, svg, textarea, video {max-width: 100%}
body {color: #444;font-family: 'Open Sans', Helvetica, sans-serif;font-weight: 300;margin: 0;text-align: center}
img {border-radius: 50%;height: 200px;margin: 0 auto;width: 200px}
a, a:visited {color: #3498db}
a:active, a:focus, a:hover {color: #2980b9}
pre {background-color: #fafafa;padding: 1rem;text-align: left}
blockquote {margin: 0;border-left: 5px solid #7a7a7a;font-style: italic;padding: 1.33em;text-align: left}
li, ol, ul {text-align: left}
p {color: #777}</style>

```